WafCrypto

Authentication System

Technical Documentation • v1.0 • 2026

.NET 8	Blazor WASM	ASP.NET Core

© 2026 WafCrypto. All rights reserved.

1. Project Overview

WafCrypto is a web-based authentication system built with Blazor WebAssembly (frontend) and ASP.NET Core Web API (backend). The system provides secure user registration and login with JWT token authentication, account lockout protection, and session management.

1.1 Tech Stack

Layer	Technology	Description
Frontend	Blazor WASM (.NET 8)	Runs in browser via WebAssembly
Backend	ASP.NET Core Web API (.NET 8)	RESTful API with JWT authentication
Database	SQLite + EF Core 8	Lightweight file-based database
Auth	JWT Bearer Token	Stateless token-based authentication
Password	BCrypt.Net	Adaptive hashing, prevents brute force

1.2 Application Sitemap

The application consists of four pages accessible from the Login entry point:

Page	Route	Description
Login	/login	Entry point — username & password form with lock protection
Register	/register	New user registration with input validation
Welcome	/welcome	Authenticated dashboard with session countdown
Lock Screen	/lock	Account locked screen with countdown timer

© 2026 WafCrypto. All rights reserved.

2. Folder Structure

2.1 Solution Structure

Folder	Description
Coding/	Root solution folder
Coding/Frontend/	Blazor WebAssembly project
Coding/Backend/	ASP.NET Core Web API project

2.2 Frontend Structure

```
Frontend/
├── Features/
│   └── AuthService.cs          # HTTP calls, JWT storage, lock countdown
├── Pages/
│   ├── Login.razor             # Route: /login
│   ├── Register.razor          # Route: /register
│   ├── Welcome.razor           # Route: /welcome
│   ├── Lock.razor              # Route: /lock
│   └── Error.razor             # Fallback error page
├── Layout/
│   ├── AuthLayout.razor        # Layout for Login & Register
│   └── EmptyLayout.razor       # Bare layout for Welcome & Lock
├── wwwroot/
│   ├── index.html              # Entry point HTML
│   └── css/app.css             # Global styles & CSS variables
└── Program.cs                  # DI registration, HttpClient config
```

2.3 Backend Structure

```
Backend/
├── Data/
│   ├── AppDbContext.cs         # EF Core DB context
│   ├── User.cs                 # User entity
│   └── Session.cs              # Session entity
├── Services/
│   └── AuthService.cs          # Business logic: Login, Register, JWT
├── Handler/
│   └── AuthHandler.cs          # Minimal API route definitions
├── Migrations/                 # EF Core auto-generated migrations
├── appsettings.json            # JWT config, connection string
└── Program.cs                  # Middleware pipeline, DI setup
```

© 2026 WafCrypto. All rights reserved.

© 2026 WafCrypto. All rights reserved.

3. Backend

3.1 Data Models

User.cs

Property	Type	Description
Id	int	Primary key, auto-increment
Username	string	Unique, lowercase, 3–30 chars
Email	string	VARCHAR(255), unique email
PasswordHash	string	BCrypt hashed password
CreatedAt	DateTime	UTC timestamp of registration
Sessions	ICollection<Session>	Navigation property — one-to-many

Session.cs

Property	Type	Description
Id	int	Primary key, auto-increment
UserId	int	Foreign key → Users.Id (CASCADE)
User	User	Navigation property — belongs to User
CreatedAt	DateTime	UTC session creation timestamp

3.2 API Endpoints

Method	Route	Auth Required	Description
POST	/api/register	No	Create new user account
POST	/api/login	No	Authenticate and return JWT token
POST	/api/logout	No	Clear server-side session state
GET	/api/me	Yes (JWT)	Return authenticated user details

3.3 Authentication Flow

- Client sends POST /api/login with { username, password } as JSON
- Backend queries DB and verifies password with BCrypt.Verify()
- On success: generates JWT token with username claim, 7-day expiry
- Returns { token, username } — client stores both in localStorage

© 2026 WafCrypto. All rights reserved.

- Protected endpoints (/api/me) require Authorization: Bearer <token> header
- JWT middleware validates token signature, issuer, and expiry automatically

3.4 Account Lockout

- Failed login attempts are tracked in-memory per username
- After 3 failed attempts: account locked for 30 seconds
- Lock expiry timestamp returned in API response as remainingSeconds
- Frontend stores lock expiry in localStorage for persistence across refreshes
- Lock screen displays countdown and auto-redirects to login when expired

© 2026 WafCrypto. All rights reserved.

4. Frontend

4.1 AuthService

AuthService is the central service class injected into all pages. It handles all communication with the backend API and manages localStorage state.

Method	Description
Login(username, password)	POST /api/login, stores token + lockExpiry to localStorage
Register(username, password)	POST /api/register with validation
Logout()	Clears token, loggedInUser, sessionExpiry from localStorage
GetCurrentUser()	GET /api/me with JWT bearer header, returns UserDto
GetLockRemainingSeconds()	Reads lockExpiry from localStorage, returns seconds remaining
SendWithToken(request)	Private helper — injects Authorization header to every request

4.2 Pages

Page	Behavior
Login.razor	Handles login form, shows error messages, redirects to /lock if account locked
Register.razor	Registration form, shows validation errors from backend
Welcome.razor	Reads user from /api/me, shows session countdown (30 min), auto-logout on expiry
Lock.razor	Reads lockExpiry from localStorage, shows countdown, auto-redirects to /login when done

4.3 localStorage Keys

Key	Set When	Used For
token	Login success	JWT bearer token sent in Authorization header
loggedInUser	Login success	Current username display
lockExpiry	3 failed attempts	Unix timestamp — lock countdown source of truth
sessionExpiry	Login success	Unix timestamp — session countdown source of truth

© 2026 WafCrypto. All rights reserved.

5. Security

5.1 Security Measures

Measure	Implementation
Password Hashing	BCrypt with adaptive cost factor — passwords never stored in plain text
SQL Injection	Entity Framework Core parameterized queries — all input automatically sanitized
Input Validation	Username: 3–30 chars, alphanumeric + _ . - only; password min 8 chars
JWT Auth	HMAC-SHA256 signed tokens, validated on every protected request
Account Lockout	3 failed attempts triggers 30-second lock, prevents brute force
CORS	Restricted to frontend origin only with AllowCredentials()
Transaction Safety	Register uses DB transaction — User + Session created atomically

5.2 JWT Configuration

- Algorithm: HMAC-SHA256
- Expiry: 7 days from login
- Claims: ClaimTypes.Name (username)
- Validation: Issuer, signing key, lifetime
- Key stored in appsettings.json — minimum 32 characters required

© 2026 WafCrypto. All rights reserved.

6. Getting Started

6.1 Prerequisites

- .NET 8 SDK
- SQLite (included via EF Core package)
- Docker (optional, for containerized deployment)

6.2 Running Locally

Backend

```
cd Backend
dotnet restore
dotnet ef database update
dotnet run
# API running at http://localhost:5278
# Swagger UI at http://localhost:5278/swagger
```

Frontend

```
cd Frontend
dotnet restore
dotnet run
# App running at http://localhost:5053
```

6.3 Docker Deployment

```
docker build -t apkauth .
docker run -p 8080:8080 -v $(pwd)/data:/data apkauth
```

The Docker image uses multi-stage build — SDK image for compilation, ASP.NET runtime image for execution. Database migrations run automatically on container startup via db.Database.Migrate().

6.4 Environment Configuration

appsettings.json (Backend) — configure before running:

```
{
  "Jwt": {
    "Key": "your-super-secret-key-min-32-chars!!",
    "Issuer": "Backend"
```

© 2026 WafCrypto. All rights reserved.

```
  },
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  }
}
```

© 2026 WafCrypto. All rights reserved.